Optimization and Machine Learning in VLSI Design Automation
Peking University, Fall 2026
Lecturer: Yibo Lin
Teaching Assistants: Shenglu Hua, Haichuan Liu
Release Date: October 26, 2026, 20:30
Due Date: November 10, 2026, 0:00

Homework 3: Partition and Floorplan

Notation

A partition is $b : V \to \{0, 1\}$. For a graph edge (u, v), its cut indicator is $[b(u) \neq b(v)]$. For a hyperedge e, its cut indicator is 1 iff e contains vertices in both blocks. All ties are broken lexicographically.

Problem 1: Partitioning (30 points)

Consider the netlist

$$NL_1 = \{n_1 = \{a, b, c\},\ n_2 = \{b, d, e, f\},\ n_3 = \{c, f, g\},\\ n_4 = \{a, g\},\ n_5 = \{d, e, h\},\ n_6 = \{f, h\}\}$$

and the initial partition $P_1 = \{a, b, c, d \mid e, f, g, h\}$.

(a) KL, 10 points. Replace every net by a clique: a net with k cells becomes an edge of weight $1/(k-1)$ between every pair of its cells, and parallel weights are summed. For a vertex v, let

$$D_v = \sum_{\substack{\{u,v\} \text{ edge} \\ b(u) \neq b(v)}} c_{uv} - \sum_{\substack{\{u,v\} \text{ edge} \\ b(u) = b(v)}} c_{uv}.$$

Exchanging u in block 0 with v in block 1 has gain

$$g(u, v) = D_u + D_v - 2c_{uv}.$$

Perform *one* KL pass:

1. Among all unlocked pairs (u, v) with $b(u) = 0$ and $b(v) = 1$, choose a pair of maximum gain. Break equal-gain ties by choosing the lexicographically smallest pair (u, v).
2. Exchange the pair, lock both vertices, and recompute D for all unlocked vertices from the new partition.
3. Repeat until every vertex is locked (four exchanges).

Record the gain, cumulative gain, and cut after each exchange. Roll back to the *first* prefix (the empty prefix is allowed) with maximum cumulative gain. Report the retained prefix and the final partition.

(b) FM, 10 points. Treat NL_1 as an unweighted hypergraph; every cell has unit area. A move is legal only if both blocks contain between 3 and 5 cells after the move. Define

$$g(v) = \text{cut}(P) - \text{cut}(P \text{ after moving } v).$$

Perform *one* FM pass:

1. After every move, recompute $g(v)$ from scratch for every unlocked cell.
2. Move the legal unlocked cell with maximum gain; break ties by cell name. Lock the moved cell.
3. Stop when no legal unlocked cell remains.

Record the moved cell, its gain, and the cut after every move. Roll back to the *first* prefix (the empty prefix is allowed) with minimum cut. Report the retained prefix and the final partition.

(c) Max-flow, 10 points. Replace every undirected edge $\{u, v\}$ of the graph from part (a), of capacity c_{uv}, by two directed arcs $u \to v$ and $v \to u$, each of capacity c_{uv}. Initially all flows are zero. Use $s = a$ and $t = h$. Run one BFS from s in the residual network, scanning adjacent vertices in alphabetical order, and stop when t is first discovered. Report the augmenting path, its bottleneck capacity, and the updated flow and residual capacity on every arc of the path.

Problem 2: Sequence Pair (30 points)

Let $P = (1, 7, 4, 5, 2, 6, 3, 8)$ and $N = (8, 4, 7, 2, 5, 3, 6, 1)$. Module i has width w_i and height h_i:

i	1	2	3	4	5	6	7	8
w_i	2	1	3	3	3	5	1	2
h_i	4	3	3	5	2	3	2	4

For two modules i, j, define

$$i \text{ left of } j \iff pos_P(i) < pos_P(j) \wedge pos_N(i) < pos_N(j),$$

$$i \text{ below } j \iff pos_P(i) < pos_P(j) \wedge pos_N(i) > pos_N(j).$$

Modules cannot rotate or overlap. Coordinates are integer lower-left corners, and all modules lie in the first quadrant.

(a) Constraint graphs, 10 points. Draw the horizontal and vertical constraint graphs on the sequence-pair grid shown in Figure 1. Label every edge with its weight (w_i for a horizontal edge leaving i, h_i for a vertical edge leaving i). Each template shows the grid, the positive/negative sequence axes, and modules 1 and 8 already placed; place the remaining modules on the grid from the sequences, then add the directed edges and their weights.

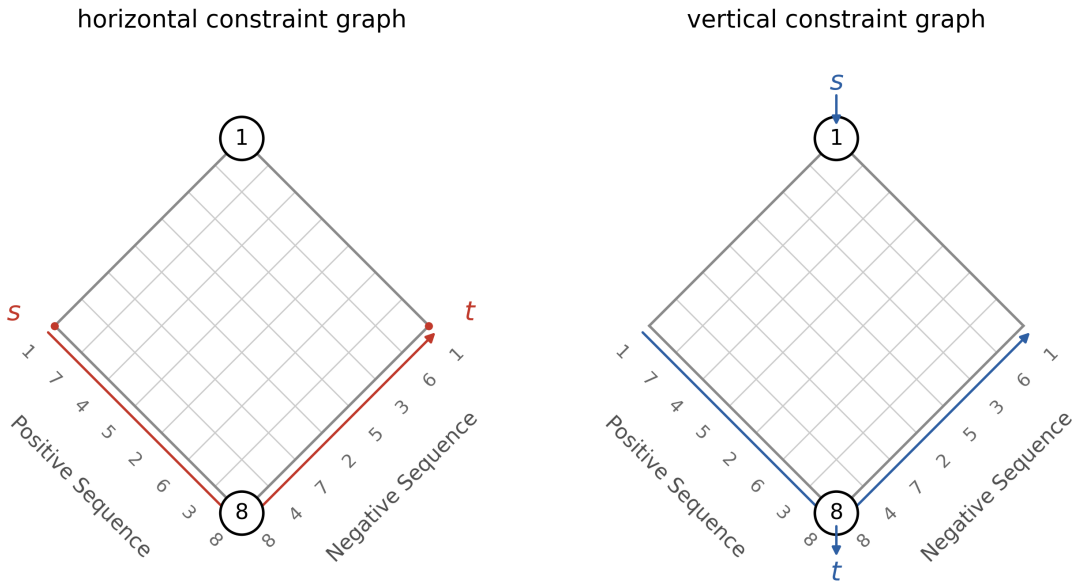


Figure 1: Blank templates for Problem 2(a): the sequence-pair grid, the labeled sequence axes, and modules 1 and 8 are drawn; place the remaining modules, add the constraint-graph edges, and label each with its weight.

(b) Relative positions, 10 points. Fill the relative-position table in Figure 2. For each module, list the modules that are to its left, to its right, above it, and below it. Hint: you may read the sets off the graphs from part (a): j is in i's left-of set exactly when the horizontal graph contains a directed path from i to j, and similarly above/below in the vertical graph.

Module	Left-of	Right-of	Above	Below
1	$\emptyset$	$\emptyset$	$\emptyset$	$\{2,3,4,5,6,7,8\}$
2				
3				
4				
5				
6				
7				
8				

Figure 2: Relative-position table

(c) Packing, 10 points. Let (x_i, y_i) be lower-left corners and define $W = \max_i(x_i + w_i)$ and $H = \max_i(y_i + h_i)$. Draw a legal floorplan minimizing WH, and report every (x_i, y_i). Solutions with the same minimum area are equivalent.

Problem 3: Macro floorplanning in OpenROAD (40 points)

The `sky130hd/chameleon` design is a real SoC with six hard macros. In the OpenROAD-flow-scripts (ORFS) flow, the `floorplan` step produces:

- `2_1_floorplan.odb`: after `initialize_floorplan`, with all six macros unplaced at $(0, 0)$;
- `2_floorplan.odb`: after OpenROAD's RTLMP macro placer and the tapcell/PDN steps.

The runner below can generate both, or you can run them yourself:

```
cd $ORFS_ROOT/flow
make DESIGN_CONFIG=designs/sky130hd/chameleon/config.mk floorplan
```

Use the OpenROAD install from Lab 0 (verified with `26Q3-1510-g6cb3f2b704`). The six macros, their masters, and their sizes in μm are:

instance	master	width	height
`RAM.genblk1[0].RAM`	`DFFRAM_4K`	1089.845	1389.36
`RAM.genblk1[1].RAM`	`DFFRAM_4K`	1089.845	1389.36
`RAM.genblk1[2].RAM`	`DFFRAM_4K`	1089.845	1389.36
`ahb_sys_0_uut.S0.CACHE`	`DMC_32x16HC`	397.61	600.00
`ahb_sys_0_uut.apb_sys_inst_0`	`apb_sys_0`	448.09	700.00
`ibex_core`	`ibex_wrapper`	600.00	800.00

The ODB stores the bracketed instance names with backslash escapes; use the exact names returned by the ODB API. The core area is $(2.3, 2.72)$–$(2680.88, 3484.32)$ μm. The flow uses a 35 μm macro halo. Macros may use only the orientations `R0`, `MX`, `MY`, and `R180`; 90-degree rotations are not allowed, so the width and height of every macro stay fixed.

(a) Implementation requirements. Build a macro floorplanner inside OpenROAD. The bundle ships a runnable skeleton `hw3.py`; the ODB reader, the simulated-annealing driver, and the JSON/Tcl writers are provided. The two algorithmic stubs are:

1. *Sequence-pair packer*: given permutations P, N and an orientation for every macro, extract the left-of/below-of relations, build the horizontal and vertical constraint DAGs, and compute the smallest feasible lower-left coordinates by longest paths in topological order. Use $(w_i + 35)$ and $(h_i + 35)$ as edge weights so adjacent macros keep the halo. The routine must run in $O(n^2)$ for n macros.
2. *Macro HPWL*: for every net with at least two macro pins, sum the half-perimeter of the bounding box of its pin centers. Pin offsets must be transformed by the chosen orientation (`R0`, `MX`, `MY`, `R180`).

Use the provided annealing driver (or replace it) to search over P, N, and orientations. Minimize macro HPWL; penalize packings that exceed the core outline, and use packed area only as a tie-breaker.

(b) Deliverables. The entry point must run as

```
python3 run_hw3.py --orfs-root /path/to/OpenROAD-flow-scripts
```

If your implementation lives in `src/`, add `--student src/hw3.py`. The runner:

1. runs `hw3.py` inside OpenROAD Python on `2_1_floorplan.odb` (the provided ODB reader needs OpenROAD's `odb` module, so `python3 hw3.py` alone cannot work — the runner executes `openroad -python ...hw3.py...` for you);
2. sources the generated `macro_placement.tcl`, then runs

   ```
   place_pins -hor_layers met3 -ver_layers met2
   global_placement -density 0.6
   detailed_placement
   ```

3. writes `flow_report.json` and validates the result with `check.py`.

Your `hw3.py` must write:

```
macro_placement.json   # macro -> x, y, orient (microns)
macro_placement.tcl    # place_macro commands sourced by the runner
```

The skeleton's `write_outputs()` already emits both files. Do not hard-code the six instance names: read the macros, dimensions, pins, and nets from the ODB.

(c) Acceptance. The bundled checker requires:

- all six macros placed inside the core, with no overlap and at least 35 μm spacing (the checker allows 1 μm for OpenROAD's placement-grid snapping);
- all orientations in {`R0`, `MX`, `MY`, `R180`};
- post-`detailed_placement` HPWL at most 730,000 μm.

For reference, the RTLMP macro positions (copied from `2_floorplan.odb` onto the same `2_1_floorplan.odb` input) run through the same placement flow give 738,243.9 μm. The checker is `check.py`; the runner invokes it automatically.

(d) Report. In your PDF, report:

1. the macro HPWL, packed width and height, the post-placement HPWL, and its ratio to the RTLMP baseline;
2. a partition analysis: split the six macros into two clusters with macro area balanced within $\pm 10\%$ of half the total area, and report the hypergraph cut (number of nets spanning both clusters). Show what cut every balanced split achieves and explain why;
3. one concrete limitation of your search or cost function, supported by a run or a small example.

Requirements

Submit a zip named `hw3-YOURSTUDENTID.zip` containing:

```
README.md
src/
hw3-YOURSTUDENTID.pdf
```

`src/` holds your implementation in any language. The README must state the exact command(s) that run your entry point and the ORFS path used.